Développement Web:
Javascript & PHP

PARTIE 2: PHP

TABLEAUX, CHAINES DE CARACTÈRES, ET FONCTIONS

PLAN

- Les chaines de caractères
- Les tableaux
- Les fonctions
- Les formulaires

Les chaines de caractères

Les chaînes de caractères

- Une chaîne de caractères est une suite de caractères alphanumériques contenus entre des guillemets simples (apostrophes) ou doubles.

```
"Ceci est une chaîne de caractères valide."
'Ceci est une chaîne de caractères valide.'
"Ceci est une chaîne de caractères invalide.'
```

- Si une chaîne contient une variable, celle-ci est évaluée, et sa valeur incorporée à la chaîne uniquement si vous utilisez des guillemets et non des apostrophes :

```
$a = 'PHP';
$b = 'MySQL';
$c = "PHP et $b";//affiche : PHP et MySQL
```

```
$d = 'PHP et $b';
/*affiche PHP et $b */
```

Les chaînes de caractères

- **Les séquences d'échappement**

Séquence	Signification
\'	Affiche une apostrophe.
\"	Affiche des guillemets.
\$	Affiche le signe $.
\\	Affiche un antislash.
\n	Nouvelle ligne (code ASCII 0x0A).
\r	Retour chariot (code ASCII 0x0D).
\t	Tabulation horizontale (code ASCII 0x09).
\[0-7] {1,3}	Séquence de caractères désignant un nombre octal (de 1 à 3 caractères 0 à 7) et affichant le caractère correspondant :echo "\115\171\123\121\114"; //Affiche MySQL.

Les chaînes de caractères

- L'opérateur PHP de **concaténation** est le point « . », qui fusionne deux chaînes littérales ou contenues dans des variables en une seule chaîne.

```
$a = "PHP";
$b = "MySQL";
$c = "Utilisez ".$a." et ".$b. " pour construire un site dynamique";
echo $c;//Utilisez PHP et MySQL pour construire un site dynamique
```

Affichage des chaînes

- Avec **echo**
  - Vous pouvez utiliser l'opérateur de concaténation entre chaque expression

```
$nom = "user";
echo "Bonjour Monsieur ". $nom. " nous sommes le ". date('d');
```

  - Ou bien séparer chaque expression par une virgule

```
echo "Bonjour Monsieur ", $nom," nous sommes le ",date('d');
```

  - Il n'est pas possible d'inclure de fonction dans une chaîne unique

```
echo "Bonjour Monsieur $nom nous sommes le date('d')" ;
```

→la variable $nom est bien évaluée, la fonction date() ne l'est pas.

Affichage des chaînes

- Avec **print**
  - quasiment similaire à echo, dans les deux cas suivants, on obtient la valeur retournée par la fonction date().

```
print ("Bonjour Monsieur $nom nous sommes le ". date('d') );
```

  - Ou bien:

```
print ("Bonjour Monsieur $nom nous sommes le "). date('d')
```

Affichage formaté

- Un affichage formaté permet d'obtenir des résultats uniformes,
- On dispose de deux fonctions pour cela, **`printf()`** et **`sprintf()`**.
- Syntaxe:

```
void printf(string "format", string $ch1, string $ch2,..... $chN)
```

  - affiche directement le contenu des chaînes $ch1, $ch2, …, $chN selon le format spécifié dans la chaîne "format"

```
string sprintf(string "format",string $ch1, string $ch2,..... $chN)
```

  - Elle retourne une chaîne unique, qui peut être composée de une ou plusieurs fois les chaînes $ch1, $ch2, …, $chN formatées selon le modèle défini dans la chaîne "format"

Caractère	Signification
%b	Interprète la chaîne $ch comme un entier et l'affiche en binaire : `$ch="89";` `printf ("En binaire $ch  s'écrit %b  ", $ch) ;` `//Affiche :En binaire 89 s'écrit 1011001`
%c	Interprète la chaîne $ch comme un entier et affiche le caractère dont le code ASCII correspond à ce nombre : `$ch="115";` `printf (" Le caractère de code $ch est %c  ", $ch);` `//Affiche: Le caractère de code 115 est s`

%d	Interprète la chaîne $ch comme un entier signé et l'affiche comme un nombre en base 10 :
	`$ch = "-25";`
	`printf ("La valeur de \$ch est %d", $ch); //Affiche -25`

%f	Interprète la chaîne $ch comme un nombre de type double et l'affiche avec sa partie décimale à 6 chiffres. Les caractères non numériques de $ch ne sont pas pris en compte :
	`$ch = "25.52 mètres";`
	`printf ("La longueur est de %f m", $ch);`
	`// Affiche: La longueur est de 25.520000 m`

%o	Interprète le chaîne $ch comme un entier et l'affiche en octal :
	`$ch = 252;`
	`printf ("En octal le nombre $ch s'écrit %o", $ch);`
	`// Affiche: En octal le nombre 252 s'écrit 374`

%s	Interprète $ch comme une chaîne et l'affiche telle quelle :
	`$ch1 = "Monsieur " ; $ch2 = " Schtroumpf" ;`
	`sprintf ("Bonjour,%s %s bravo !",$ch1 $ch2);`
	Équivaut à :
	`echo "Bonjour $ch1 $ch2, bravo !";`

%x
ou
%X
	Interprète la chaîne $ch comme un entier et l'affiche en hexadécimal en minuscules (%x) ou (%X) :
	`$ch ="252756";`
	`printf ("En hexadécimal $ch s'écrit %x ", $ch) ;`
	`// Affiche: En hexadécimal 252756 s'écrit 3db54`

Mise en forme des chaînes

- Principales fonctions de mise en forme d'une chaîne de caractères

string strtolower(string $chaine)	Retourne la chaîne avec tous les caractères en minuscules.
string strtoupper(string $chaine)	Retourne la chaîne avec tous les caractères en majuscules.
string ucwords(string $chaine)	Retourne la chaîne avec toutes les initiales des mots qui la composent en majuscules.
string ucfirst(string $chaine)	Retourne la chaîne avec uniquement la première lettre en majuscule.
string ltrim(string $chaine [,string liste])	Renvoie la chaîne nettoyée des espaces situés en début de chaîne.
string rtrim(string $chaine [,string liste])	Renvoie la chaîne nettoyée des espaces situés en fin de chaîne.
string trim(string $chaine [,string liste])	Supprime les espaces situés en début et en fin de chaîne.
string wordwrap(string $chaine [, int N [,string car [,boolean coupe]]])	Permet d'afficher un texte long avec une largeur maximale déterminée. Le paramètre N définit cette largeur. Le paramètre car contient la chaîne à insérer dans $chaine tous les N caractères. Le paramètre booléen coupe permet, s'il vaut TRUE, d'effectuer une césure des mots dont la longueur dépasse N caractères.

Mise en forme des chaînes

- Exemples:

```php
<?php
    $nom = "ENgelS" ;
    $prenom = "jEan " ;
    $adresse = "21, rue compoinT" ;
    $ville = "75018 pAris" ;
    $mail = "ENGELS@funPHP.Com" ;
    $prenom = ucfirst (strtolower ($prenom )) ;
    $nom = strtoupper ($nom) ;
    $adresse = ucwords(strtolower ($adresse )) ;
    $ville = strtoupper ($ville);
    $mail = strtolower ($mail);
    echo "Mes coordonnées <br>";
    echo $prenom, $nom, "<br>";
    echo $adresse, "<br>" ;
    echo $ville, "<br>" ;
?>
```

```
Mes coordonnées
Jean ENGELS
21, rue Compoint
75018 PARIS
```

Mise en forme des chaînes

- Exemple

```
<?php
    $a = "...Jean " ;
    $b = "Dupont___";
    echo $a,$b,"<br />";
    echo trim($a,' .')," ",rtrim($b,' _');
?>
```

```
...Jean Dupont___
Jean Dupont
```

- Exemple

```
<?php
    echo wordwrap($ch,30,"<br>",1)
```

  - le contenu de la chaîne $ch s'affiche dans une colonne de 30 caractères de largeur. Dans le code source HTML, vous trouvez donc un élément
 tous les 30 caractères.

Recherche de sous-chaines

- Les principales fonctions:
  - Voir Tableau 8
- Exemples:
  - Voir Tableau 9

Comparaison de chaines

- Les principales fonctions:
  - Voir Tableau 10
- Exemples:
  - Voir Tableau 11

Transformation de chaines en tableaux

- Les principales fonctions:
  - Voir Tableau 12

Les tableaux

Les tableaux

- Comme dans tous les langages, les tableaux sont une structure de données incontournables.
- La déclaration d'un tableau se fait comme une variable (préfixé par $)
- Création à l'aide de la fonction `array()`
- L'indice du tableau se trouve entre `[` et `]`
- Exemples

```
$MonTableau[0] = 1;
$MonTableau[] = 23; // = $MonTableau[1] = 23;
$MonTableau[] = 18; // = $MonTableau[2] = 18;
$MonTableau[0][0] = 1; // Tableau à 2 dimensions
```

Les tableaux

- Les éléments d'un tableau peuvent appartenir à des types différents

```
// Déclaration d'un entier et chaine
$tab = array($variable, "texte", 153, 56);
```

- L'index d'un tableau en PHP commence de 0
- Les éléments d'un tableau peuvent avoir des indices négatifs. Un indice négatif permet d'accéder aux éléments à partir de la fin du tableau.
  - Le dernier élément du tableau non vide est $tab[-1]
- Pas de limites supérieures pour les tableaux
- Pas besoin de préciser à l'avance le nombre d'éléments du tableau

Tableau à indices numériques

- Accéder aux éléments par l'intermédiaire de numéros

```
$tableau = array(valeur0, valeur1,..., valeurN);
$jour = array("Dimanche", "Lundi", "Mardi", "Mercredi",
                "Jeudi", "Vendredi", "Samedi");
$note = array(20, 15, 12.6, 17, 10, 20, 11, 18, 19);

$tableau[indice] = valeur;
$jour[3] = "Mercredi";
$note[0] = 20;

$variable = $tableau[indice];
$JJ = $jour[6]; // affecte "Samedi" à $JJ
echo $note[1] + $note[5];
```

Tableau à indices numériques

- Parcours

```
foreach ($tableau as $valeur) {

  // instructions utilisant $valeur
}
```

```
$jour = array("Dimanche", "Lundi", "Mardi", "Mercredi", "Jeudi",
"Vendredi", "Samedi");
$i = 0;
foreach($jour as $JJ)
{
   echo "La cellule ". $i . " : " . $JJ . "<br>";
   $i++;
}
```

Tableau associatif

- Les éléments sont référencés par des chaînes de caractères associatives en guise de nom: la clé d'index
- Pour créer un tableau associatif, il faut donner pour chaque élément, le couple : (clé => valeur)

```php
<?php
    $tab = array(
        "prenom" => "Cyril",
        "ville" => "Paris",
        "specialite" => "informatique"
        );
?>
```

Clé	Valeur
prenom	Cyril
ville	Paris
spécialité	informatique

Tableau associatif

- Exemple:

```
$tableau = array(ind0 => val0, ind1 => val1,..., indN => valN);
$jour = array("Dimanche" => 1, "Lundi" => 2, "Mardi" => 3,
              "Mercredi" => 4, "Jeudi" => 5,
              "Vendredi" => 6, "Samedi" => 7);

$tableau["indice"] = valeur;
$jour["Dimanche"] = 7 ;
$jour["Mercredi"] = "Le jour des enfants" ;

$variable = $tableau["indice"];
$JJ = $jour["Vendredi"]; // affecte 6 à $JJ
echo $jour["Lundi"]; //retourne la valeur 2
```

Tableau associatif

- Parcours:

```
foreach ($tableau as $cle=>$valeur) {

   // instructions utilisant $valeur
}
```

```
$jour = array("Dimanche" => 7, "Lundi" => 1,   "Mardi" => 2,
"Mercredi" => 3, "Jeudi" => 4, "Vendredi" => 5, "Samedi" => 6);

foreach($jour as $sJJ => $nJJ)

{

    echo "Le jour de la semaine n° ".$nJJ.":".$sJJ."<br>";

}
```

Déboguer un tableau

- PHP print_r() et var_dump()
  - Ces fonctions affichent la vraie structure du tableau : structure lisible

```php
<?php
    $colors = array('rouge', 'vert', 'bleu');
    echo '<pre>';
    print_r($colors);
    var_dump($colors);
    echo '</pre>';
?>
```

```
Array
(
    [0] => rouge
    [1] => vert
    [2] => bleu
)
```

```
array (size=3)
  0 => string 'rouge' (length=5)
  1 => string 'vert' (length=4)
  2 => string 'bleu' (length=4)
```

Fonctions sur les tableaux

- explode($separateur, $tableau) : convertit une chaine de caractères en tableau itératif.
- implode($separateur, $tableau) : convertit un tableau en chaine de caractères.
- count($tableau) /sizeof($tableau) : renvoie la taille du tableau (le nombre d'objets qu'il contient).
- array_merge($monTableau1, $monTableau2, $monTableau3) : fusionne plusieurs tableaux.
- array_walk($tableau, fonction) exécute une fonction sur chaque élément (clé ou valeur).

Recherche dans les tableaux

- in_array ($élement, $tableau): recherche de présence par valeur. Renvoie un booléen si l'élément est trouvé.
- Array_search ($élement, $tableau): recherche de position par valeur. Renvoie la clé de l'élément trouvé, ou NULL sinon.
- array_keys($tableau, $élement) : renvoie un tableau contenant toutes les clés du tableau en paramètre. De plus, si une valeur est définie en paramètre deux, le résultat ne contient que les clés associées à celle-ci.
- array_key_exists($cle, $tableau) : renvoie "vrai" si la clé est dans le tableau.

Tri

- array_multisort($tableau, SORT_ASC) permet de trier un tableau dans l'ordre croissant de ses valeurs.
- sort($tableau) : trie le tableau par valeurs croissantes, en recréant des clés numériques.
- asort($tableau) : trie le tableau par valeurs croissantes, en conservant les clés associées.
- usort($tableau, fonction) : trie selon une fonction donnée.
- ksort($tableau) : trie par clés croissantes.

Exemples

- **Exemples :**
  - Boucle foreach: Voir Tableau 14
  - Ajout/suppression dans un tableau: Voir Tableau 15
  - Intersection/différence entre deux tableaux: Voir Tableau 16
  - Tri tableau indicé: Voir Tableau 17 &18
  - Tri tableau associatif: Voir Tableau 19
  - Sélection des éléments: Voir Tableau 20
  - Appliquer une fonction aux éléments: Voir Tableau 21

Les fonctions

Les fonctions

- Déclaration

```
function nom_fonction($arg1, $arg2, …$argn)
{
    bloc d'instructions ;
    //fin du corps de la fonction
    return $resultat ;
}
```

- Valeur par défaut

```
function nom_fonction($arg1='valeur_par_defaut', …,$argn)
{
    bloc d'instructions ;
}
```

- Appel de fonction

```
nom_fonction(argument1, argument2,…,$argn);
```

Les fonctions

- Exemple

```
function dire_texte($qui, $texte='Bonjour'){
     if(empty($qui)){
          return FALSE;
     }
     else{
          echo "$texte $qui";
          return TRUE;}
}
//appel de la fonction
dire_texte('cher public', 'bienvenue');
//Utilisation de la valeur par défaut
dire_texte('cher public');
```

Les fonctions

- **Passage de paramètres**
- Par défaut, PHP fait un passage **par recopie**
  - La valeur utilisée par la fonction n'est donc pas celle donnée en argument mais une copie
  - Si vous la modifiez à l'intérieur de la fonction, cela n'aura pas d'influence dehors

```php
<?php
    function ajouter_cinq($nombre){
        $nombre += 5;
        return $nombre;}
    $mon_entier = 15;
    echo ajouter_cinq($mon_entier); //affichera 20
    echo $mon_entier; //affichera 15
?>
```

Les fonctions

- **Passage de paramètres par référence**
  - On fait référence à la variable dans le programme appelant et tout ce qu'on fait sur la variable est reportée au niveau du programme appelant
  - Pour cela, il faut accompagner le paramètre d'appel de **"&"**
  - Dans ce type d'opérations, vous travaillez directement sur la variable d'origine, il n'y a pas de recopie
    - Il n'est plus nécessaire de retourner une valeur

Les fonctions

- **Passage de paramètres par référence**
  - Exemple

```php
<?php
    function ajouter_cinq($nombre){
            $nombre += 5;
            }
    $mon_entier = 15;
    ajouter_cinq(&$mon_entier);
    echo $mon_entier; //affichera 20
?>
```

Exemples

- **Exemples :**
  - Appel de fonctions: Voir Tableau 25
  - Plusieurs valeurs de retour: Voir Tableau 26
  - Paramètres par défaut: Voir Tableau 27
  - Nombre variables de paramètres : Voir Tableau 28
  - Portée de variables: Voir Tableau 29

Les formulaires

Les formulaires

- Dans un contexte Web, les données échangées avec le système se font à travers des formulaires
- Les formulaires HTML sont la méthode la plus simple pour avoir des interactions avancées avec l'utilisateur
- Ils permettent, par exemple, de :
  - Créer un espace sécurisé
  - Donner aux clients la possibilité de modifier eux mêmes leurs infos
  - Interagir avec le visiteur en lui demandant des informations complémentaires…

Les formulaires

- Création : balise <form>

```
<form action='reception_formulaire.php' method='GET ou POST '>
    <!-- différents champs -->
</form>
```

- action : désigne la page vers laquelle seront envoyées les informations rentrées par l'utilisateur une fois le bouton d'envoi actionné
- Method: définit le mode d'envoi des informations au serveur: GET ou POST
  - PHP associe à ces deux variables deux tableaux $_GET et $_POST pour récupérer les données passées
  - Prenez soin de noter les noms (attribut name) des données du formulaire, car ils représentent les "clés" dans le tableau associatif

Les formulaires

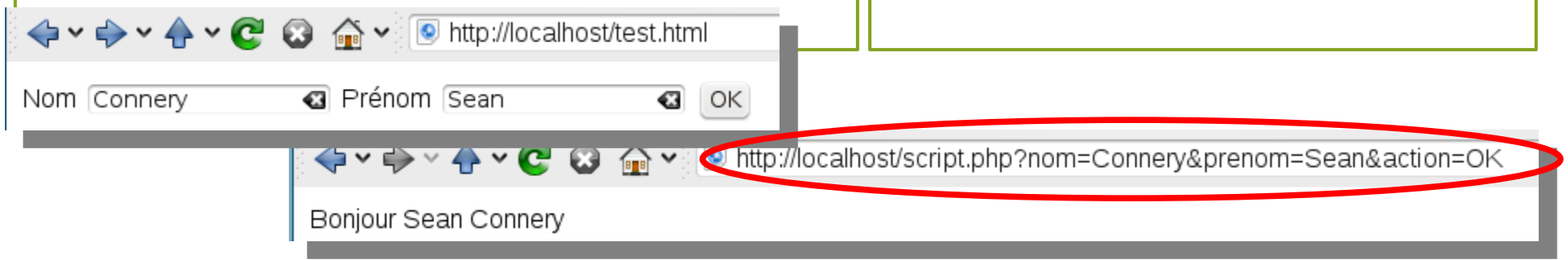
- Méthodes d'envoi GET

HTML

```
<form action="script.php"
      method="get">
Nom
<input type="text" name="nom">
Prénom
<input type="text" name="prenom">
<input type="submit"
   name="action" value="OK">
</form>
```

PHP

```
<html>
  ...
Bonjour
<?php
   echo $_GET["prenom"].
          " ".$_GET["nom"];
 ?>
</html>
```



Les formulaires

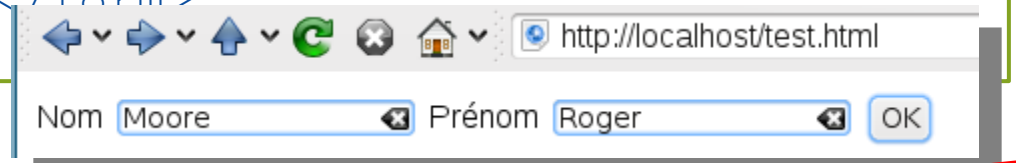
- Méthodes d'envoi POST

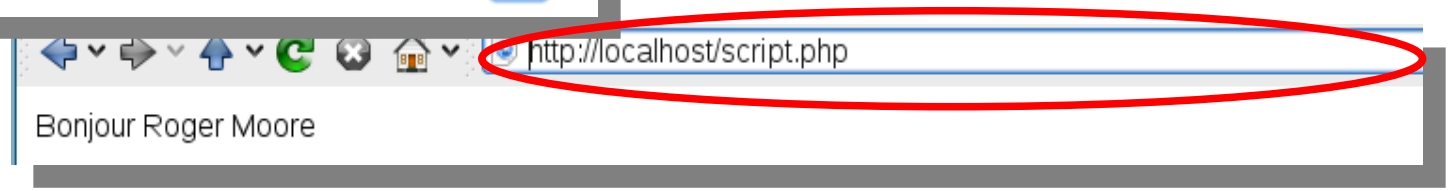
HTML

```
<form action="script.php"
      method="post">
Nom
<input type="text" name="nom">
Prénom
<input type="text" name="prenom">
<input type="submit"
   name="action" value="OK">
</form>
```

PHP

```
<html>
  ...
Bonjour
<?php
   echo $_POST["prenom"].
          "
".$_POST["nom"];
 ?>
</html>
```





Les formulaires

- **Champ de texte**

```
<form action="<?= $_SERVER["PHP_SELF"] ?>" method="post">
    <input name="mon_champ" type="text"/>
    <input name="valider" type="submit" value="OK"/>
</form>

<?php
    if (isset($_POST['mon_champ'])) {
        echo "Votre champ contenait : ". $_POST['mon_champ'];
    }
?>
```

Les formulaires

- **Les boutons radio**

```
<form action="" method="post">
<input type="radio" name="mon_champ" value="Option1"/>Option1 <br>
<input type="radio" name="mon_champ" value="Option2"/>Option2 <br>
<input type="radio" name="mon_champ" value="Option3"/>Option3 <br>
<input type="submit" value="OK"/>
</form>

<?php
    if (isset($_POST['mon_champ'])) {
        echo "Votre champ contenait : ". $_POST['mon_champ'];
    }
?>
```

Les formulaires

- **Les cases à cocher**

```
<form action="" method="post">
 <input type="checkbox" name="mon_champ[]" value="Opt1"/> Option1<br>
 <input type="checkbox" name="mon_champ[]" value="Opt2"/> Option2<br>
 <input type="checkbox" name="mon_champ[]" value="Opt3"/> Option3<br>
 <input type="submit" value="OK">
</form>
<?php
    if (isset($_POST['mon_champ'])) {
        echo "Vous avez choisi :";
        for ($i = 0; $i < count($_POST['mon_champ']); $i++) {
            echo "<br>" . $_POST['mon_champ'][$i];   }
    }
?>
```

Les formulaires

- **Les listes déroulantes à sélection simple**

```
<form action="" method="post">
<select name="mon_champ">
  <option>Option 1</option>
  <option>Option 2</option>
  <option>Option 3</option>
</select>
<input type="submit" value="OK">
</form>
<?php
    if (isset($_POST['mon_champ'])) {
        echo "Vous avez choisi :".$_POST['mon_champ'];
    }
?>
```

Les formulaires

- **Les listes déroulantes à sélection simple**

```
<form action="" method="post">
<select name="mon_champ[]" multiple>
 <option>Option 1</option>
 <option>Option 2</option>
 <option>Option 3</option>
</select>
<input type="submit" value="OK">
</form>
<?php
    if (isset($_POST['mon_champ'])) {
        for ($i = 0; $i < count($_POST['mon_champ']); $i++){
            echo "<br>" . $_POST['mon_champ'][$i];   }
    }
?>
```

Transfert de fichier vers le serveur

- Il ne s'agit plus de transmettre une ou plusieurs valeurs scalaires au serveur mais l'intégralité d'un fichier, lequel peut avoir une taille importante et un type quelconque.
- Par rapport au transfert de données, le transfert de fichiers présente un problème de sécurité pour le site puisque des fichiers vont être enregistrés et éventuellement exécutés sur le serveur.
  - Pour cela, utiliser l'attribut **`accept`** de l'élément **`<input>`.**
  - Cet attribut peut prendre un grand nombre de valeurs, correspondant aux types MIME des fichiers acceptés, par exemple "image/gif", etc.
- L'élément <form> doit avoir l'attribut method à la valeur post et l'attribut enctype à la valeur multipart/form-data.

Transfert de fichier vers le serveur

- Il est possible de limiter la taille des fichiers à télécharger en ajoutant au formulaire un champ caché dont l'attribut value contient la taille maximale admise en octet.

```
<input type="hidden" name="MAX_FILE_SIZE" value="100000" />
```

- L'utilisation de ce champ n'est pas obligatoire puisque le fichier **php.ini** du serveur contient la directive "**upload_max_filesize**", dont la valeur est un entier indiquant la taille maximale en octet admise par défaut par le serveur.
- L'hébergeur peut toutefois définir une valeur différente, qu'on peut vérifier à l'aide de la fonction **`phpinfo()`**.

Transfert de fichier vers le serveur

- **Exemple**

```
<form action="" method="post" enctype="multipart/form-data">
 <input type="hidden" name="MAX_FILE_SIZE" value="100000" >
 Fichier <input type="file" name="fich" accept="image/gif" ><br>
 Clic!  <input type="submit" value="Envoi">
</form>
```

Transfert de fichier vers le serveur

- **Exemple**

```php
<?php
if(isset($_FILES['fich'])) {
 echo "Taille maximale autorisée :",$_POST["MAX_FILE_SIZE"]," octets<hr>";
 echo "<b>Clés et valeurs du tableau \$_FILES </b><br>";
 foreach($_FILES["fich"] as $cle => $valeur){
   echo "clé : $cle , valeur:$valeur <br>";
 }
```

Clés et valeurs du tableau $_FILES
clé : name , valeur : Sky-Backgrounds-pictures-768x432.jpg
clé : type , valeur : image/jpeg
clé : tmp_name , valeur : C:\wamp64\tmp\phpC1AA.tmp
clé : error , valeur : 0
clé : size , valeur : 34188

Le transfert est réalisé !

Transfert de fichier vers le serveur

```
        //Enregistrement et renommage du fichier
        $result=move_uploaded_file($_FILES["fich"]["tmp_name"],"imagephp.gif");
        if($result==TRUE)
            {echo "<hr><b>Le transfert est réalisé !</b>";}
        else
        {echo "<hr> Erreur de transfert n°",$_FILES["fich"]["error"];}
}
?>
```

Exemples

- **Exemples :**
  - Récupération des données d'un formulaire:
    - Valeurs uniques: Voir Tableau 22-1 et 22-2
    - Valeurs multiples: Voir Tableau 22-3
  - Transfert de fichiers: Voir Tableau 23
  - Gestions de boutons d'envoi multiple Voir Tableau 24

Questions?